
ESTANDARTE LED INTERACTIVO

SIMULACIÓN

Escuela Profesional de Ingeniería Mecánica Eléctrica
Universidad Nacional del Altiplano — Puno, Perú

Parada Universitaria 2026

Proyecto:	Estandarte LED Interactivo — Simulación
Integrantes:	▪ Steven Alex Mayta Lampa ▪ Dilmar H. Siguayro Coila ▪ Aquiles T. Ramos Yapo ▪ Renzo G. Mamani Galindo ▪ Martin Calla Quispe ▪ Abel Y. Rivera Quispe
Responsable de asignatura:	Chura Acero Julio Fredy
Fecha:	4 de agosto de 2026
Lugar:	Puno, Perú

Índice

1. Resumen ejecutivo	1
2. Antecedentes y necesidad	1
2.1. Configuración visual prevista	2
3. Objetivos	2
3.1. Objetivo general	2
3.2. Objetivos específicos	2
4. Enlaces rápidos y acceso a la simulación	3
5. Software recomendado	3
5.1. Herramienta principal: Wokwi	3
5.2. Visual Studio Code + PlatformIO	4
5.3. Herramientas complementarias	4
6. Instalación paso a paso del entorno de simulación	5
6.1. Ruta rápida: Wokwi en navegador	5
6.2. Ruta profesional: VS Code + PlatformIO + Wokwi	5
7. Vista visual de la simulación	6
8. Alcance de la simulación electrónica integrada	6
9. Distribución de pines y tendido de señales	7
9.1. Código de colores de cableado	7
9.2. Reglas de tendido físico	8
10. Modos de iluminación	8
10.1. Secuencia principal propuesta	8
11. Arquitectura física final de tres ESP32	9
11.1. Razón para utilizar ESP-NOW	9
11.2. Estructura del mensaje	9
11.3. Control desde el celular	9
12. Sistema de potencia y baterías	10
12.1. Arquitectura por módulo	10
12.2. Ecuaciones de dimensionamiento	10
12.3. Escenarios preliminares	10
12.4. Opciones de batería a evaluar	11

13.Diseño mecánico y distribución física	11
13.1. Criterios de diseño	11
13.2. Materiales candidatos	11
14.Plan de trabajo integral	12
14.1. Estructura de desglose del trabajo	12
14.2. Cronograma referencial de 20 días	12
14.3. Matriz RACI preliminar	14
15.Lista preliminar de materiales	14
15.1. Formato de presupuesto	15
16.Plan de pruebas y criterios de aceptación	15
16.1. Registro de resultados	16
17.Riesgos y medidas de control	16
18.Organización del repositorio y control de versiones	17
18.1. Reglas de trabajo	17
19.Manual operativo preliminar	18
19.1. Antes de la parada	18
19.2. Durante la parada	18
19.3. Después de la actividad	18
20.Entregables y criterios de cierre	18
21.Conclusiones y decisión recomendada	19
A. Anexo A: archivo PlatformIO	19
B. Anexo B: configuración Wokwi local	19
C. Anexo C: protocolo común de tres ESP32	19
D. Anexo D: firmware del ESP32 central	20
E. Anexo E: receptor Mecánica	21
F. Anexo F: receptor Eléctrica	22
G. Anexo G: enlaces oficiales	23

Índice de figuras

1.	Códigos QR para acceder a las herramientas y documentación principal.	3
2.	Cronograma referencial de 20 días, ajustado a la fecha final del 24 de agosto de 2026. Debe confirmarse con la fecha oficial y la disponibilidad del equipo.	13

Índice de cuadros

1.	Composición funcional de los tres estandartes	2
2.	Capacidades de Wokwi aplicadas al proyecto	4
3.	Software complementario y función dentro del proyecto	4
4.	Elementos incluidos en el proyecto Wokwi	6
5.	Pinout de la simulación integrada	7
6.	Catálogo de modos programados y propuestos	8
7.	Campos principales del protocolo propuesto	9
8.	Escenarios referenciales de corriente a 5 V	10
9.	Fases, actividades y entregables	12
10.	Asignación de responsabilidades	14
11.	BOM preliminar para planificación	14
12.	Cuadro para completar cotizaciones	15
13.	Matriz de verificación	15
14.	Matriz preliminar de riesgos	16

1 Resumen ejecutivo

Finalidad del proyecto

El proyecto busca desarrollar tres estandartes móviles e independientes para representar a la Escuela Profesional de Ingeniería Mecánica Eléctrica durante la parada universitaria. El módulo izquierdo llevará la palabra **MECÁNICA**; el módulo central, de mayor ancho, incluirá el nombre institucional y un símbolo de Zeus; el módulo derecho llevará la palabra **ELÉCTRICA**. La iluminación se realizará con LED direccionables WS2812, controladores ESP32, efectos dinámicos, sincronización inalámbrica y una interfaz de control desde teléfono móvil.

El trabajo se organiza en dos niveles. El primero es una **simulación integrada**, en la que un ESP32 virtual controla tres zonas LED, una pantalla OLED, botones y entradas analógicas. El segundo es la **arquitectura física final**, donde cada estandarte tendrá su propio ESP32 y batería, mientras el módulo central actuará como coordinador mediante ESP-NOW y alojará una página web local mediante Wi-Fi.

- Documento editable en $\text{\LaTeX}$ y PDF compilado.
- Simulador visual interactivo en HTML con vistas de estandarte, circuito y red inalámbrica.
- Proyecto integrado listo para importar en Wokwi.
- Proyecto local para Visual Studio Code + PlatformIO + Wokwi.
- Código base para tres ESP32: central, Mecánica y Eléctrica.
- Diagramas, enlaces, códigos QR, pinout, cronograma, pruebas, riesgos y presupuesto preliminar.

Wokwi permite simular el ESP32, GPIO, ADC, I2C, Wi-Fi y dispositivos WS2812. Sin embargo, no admite varios microcontroladores conectados dentro de un único proyecto. Por ello, la simulación inicial integra las tres zonas en un solo ESP32, mientras la comunicación ESP-NOW debe verificarse posteriormente con tres placas físicas o con varias instancias separadas del simulador. [3, 2]

2 Antecedentes y necesidad

El repositorio original propone un estandarte reactivo al sonido mediante ESP32, micrófono MAX4466, tiras WS2812 y scripts de análisis de audio. Para la parada universitaria se requiere ampliar el alcance y adaptar la propuesta a tres estructuras móviles que no estarán unidas por cables. Esto obliga a incorporar:

- controladores y baterías independientes;
- sincronización inalámbrica;
- control desde celular sin depender de internet;
- protección eléctrica y limitación de corriente;
- diseño mecánico ligero, desmontable y resistente a vibraciones;
- simulación previa del firmware, conexiones y efectos visuales.

2.1 Configuración visual prevista

Cuadro 1: Composición funcional de los tres estandartes

Módulo	Orientación	Contenido visual	Efectos principales
Mecánica	Vertical	Palabra MECÁNICA, engranajes o elementos de máquinas	Barridos, rotación simulada, pulso naranja/dorado
Centro	Ancho y principal	Nombre de la escuela, Zeus, rayo y emblema institucional	Descarga central, secuencia coordinada, página web y sonido
Eléctrica	Vertical	Palabra ELÉCTRICA, rayos, bobinas o circuitos	Chispas, descarga azul/cian, flujo ascendente

3 Objetivos

3.1 Objetivo general

Diseñar, simular y validar un sistema modular de iluminación dinámica para tres estandartes móviles de Ingeniería Mecánica Eléctrica, con control inalámbrico, operación autónoma, protección eléctrica y capacidad de interacción desde un teléfono móvil.

3.2 Objetivos específicos

1. Representar gráficamente el ESP32, drivers, entradas, tiras LED, matriz central y cableado en Wokwi.
2. Programar efectos diferenciados para Mecánica, Zeus y Eléctrica.
3. Crear una simulación visual local que permita explicar el funcionamiento sin depender del hardware.
4. Diseñar la red de tres ESP32 mediante ESP-NOW y el control web local mediante Wi-Fi.
5. Definir el tendido de potencia, señales, protecciones y conectores por módulo.
6. Establecer cálculos preliminares de consumo y autonomía.
7. Organizar las actividades en un cronograma con responsables, entregables y criterios de aceptación.
8. Preparar una lista de materiales y una matriz de pruebas antes de construir el prototipo.

4 Enlaces rápidos y acceso a la simulación

Acceso inmediato

- Abrir un proyecto nuevo de ESP32 en Wokwi.
- Abrir el repositorio base en GitHub.
- Instalar y usar PlatformIO IDE para VS Code.
- Consultar la API oficial ESP-NOW.
- Abrir el simulador visual local incluido en el ZIP.



(a) Wokwi ESP32



(b) Repositorio base



(c) PlatformIO



(d) ESP-NOW

Figura 1: Códigos QR para acceder a las herramientas y documentación principal.

Los archivos `sketch.ino`, `diagram.json` y `libraries.txt` están listos para importar. No se incluye un enlace público único de Wokwi porque la publicación de un proyecto requiere guardarlo desde una cuenta de Wokwi. Una vez importado y guardado, Wokwi generará un enlace compatible.

5 Software recomendado

5.1 Herramienta principal: Wokwi

Wokwi es la herramienta prioritaria para comenzar porque ejecuta firmware real del ESP32 y permite observar gráficamente tiras y matrices WS2812, botones, potenciómetros, OLED, conexiones y monitor serial. También ofrece simulación Wi-Fi y compatibilidad con proyectos Arduino, ESP-IDF y firmware compilado. [1, 2, 4, 5]

Cuadro 2: Capacidades de Wokwi aplicadas al proyecto

Capacidad	Aplicación	Estado
ESP32 DevKit V1	Ejecución del firmware, GPIO, ADC, I2C y monitor serial	Simulado
WS2812 Strip	Representación de Mecánica y Eléctrica	Simulado
WS2812 Matrix	Representación de Zeus y panel central	Simulado
Potenciómetros	Sonido y brillo simulados	Simulado
Botones	Selección de modo, paleta y secuencia	Simulado
OLED SSD1306	Estado del sistema	Simulado
Wi-Fi	Servidor web y pruebas de red con gateway	Simulable
Bluetooth	Control directo por Bluetooth	No simulado
Varios ESP32 en un proyecto	Red física completa	No soportado
Batería y caída de tensión	Autonomía y comportamiento eléctrico real	Cálculo/prueba física

5.2 Visual Studio Code + PlatformIO

PlatformIO ofrece un entorno de trabajo ordenado para compilar, instalar bibliotecas, abrir el monitor serial, manejar varios firmwares y mantener el proyecto bajo control de versiones. La extensión Wokwi para VS Code utiliza los archivos `wokwi.toml` y `diagram.json` para ejecutar la simulación local. [9, 7, 8]

5.3 Herramientas complementarias

Cuadro 3: Software complementario y función dentro del proyecto

Software	Función	Uso recomendado
Wokwi Web	Simulación inmediata en navegador	Primera validación de animaciones y cableado lógico
VS Code + PlatformIO	Desarrollo local, compilación y gestión de bibliotecas	Firmware integrado y firmware de tres nodos
Wokwi para VS Code	Simulación del binario compilado e IoT Gateway	Página web local y flujo de trabajo avanzado
KiCad	Esquemático eléctrico, conectores y PCB distribuidora	Ingeniería de detalle
EasyEDA	Alternativa web para esquema y PCB	Cuando se requiera fabricación rápida
Proteus	Esquema tradicional y pruebas de circuitos básicos	Complemento académico; no es la mejor opción para cientos de WS2812
SketchUp	Volumen, dimensiones y ubicación de componentes	Diseño mecánico preliminar
SolidWorks	Ensamble, soporte, peso y fabricación	Diseño mecánico definitivo
Blender	Render, video y presentación visual	Demostración de la apariencia final

Software	Función	Uso recomendado
Git/GitHub	Versionado y colaboración	Control de cambios y respaldo del proyecto

6 Instalación paso a paso del entorno de simulación

6.1 Ruta rápida: Wokwi en navegador

1. Abrir <https://wokwi.com/projects/new/esp32>.
2. Crear o iniciar sesión para guardar el proyecto y obtener un enlace compartible.
3. Reemplazar el contenido de `sketch.ino` con el archivo incluido en `wokwi_integrado`.
4. Abrir `diagram.json` y reemplazarlo con el diagrama incluido.
5. Crear `libraries.txt` con las bibliotecas indicadas.
6. Presionar *Start Simulation*.
7. Mover los potenciómetros de sonido y brillo.
8. Pulsar MODO, COLOR y SECUENCIA o utilizar las teclas M, C y S.
9. Revisar la OLED y el monitor serial.
10. Guardar el proyecto para que Wokwi genere su enlace público.

6.2 Ruta profesional: VS Code + PlatformIO + Wokwi

1. Instalar Visual Studio Code.
2. Buscar e instalar la extensión oficial *PlatformIO IDE*.
3. Instalar la extensión *Wokwi Simulator*.
4. Abrir la carpeta `platformio_simulacion`.
5. Ejecutar *PlatformIO: Build*.
6. Verificar que se generen `firmware.bin` y `firmware.elf`.
7. Ejecutar *Wokwi: Start Simulator*.
8. Para un servidor web futuro, activar el reenvío de puerto en `wokwi.toml`. [8]

Listing 1: Configuración incluida en `wokwi.toml`

```

1 [wokwi]
2 version = 1
3 firmware = '.pio/build/esp32dev/firmware.bin'
4 elf = '.pio/build/esp32dev/firmware.elf'
5
6 # Etapa posterior: servidor web virtual
7 # [[net.forward]]
8 # from = 'localhost:8180'
9 # to = 'target:80'
```

7 Vista visual de la simulación

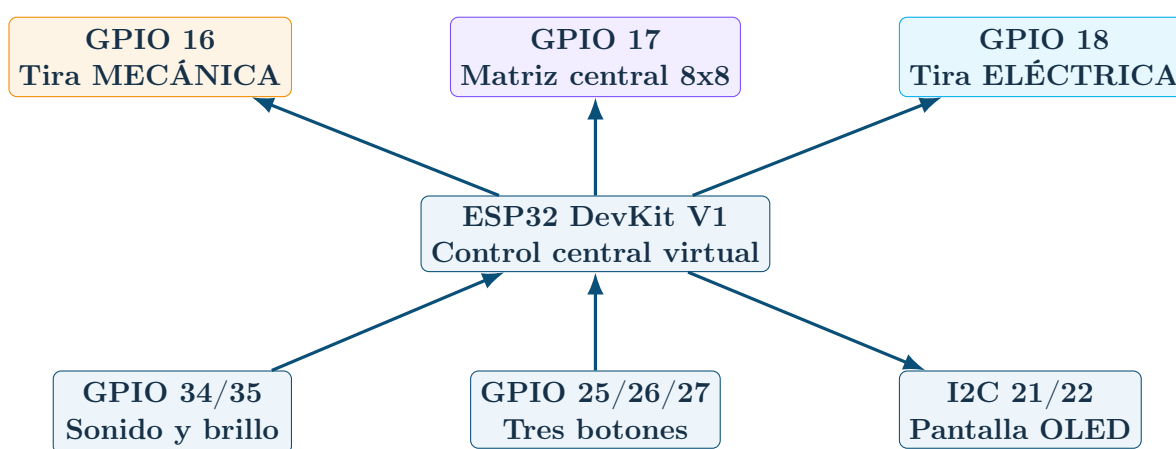
El paquete incluye el simulador visual local (`simulacion_visual/Simulador_Visual_Estandarte_IME.html`), que ofrece las cuatro vistas descritas a continuación.

El archivo HTML ofrece cuatro vistas:

1. **Estandartes iluminados:** permite cambiar modo, brillo, sonido y ejecutar la secuencia coordinada.
2. **Circuito y drivers:** muestra batería, convertidor, ESP32, adaptador lógico y zonas WS2812.
3. **Comunicación inalámbrica:** explica la relación entre el celular, el ESP32 central y los laterales.
4. **Enlaces de trabajo:** acceso a Wokwi, documentación y repositorio.

Descomprimir el ZIP y abrir `simulacion_visual/Simulador_Visual_Estandarte_IME.html` con Chrome, Edge o Firefox. No requiere instalación ni internet para las animaciones; solo los enlaces externos necesitan conexión.

8 Alcance de la simulación electrónica integrada



Cuadro 4: Elementos incluidos en el proyecto Wokwi

N.º	Elemento	Cant.	Función
1	ESP32 DevKit V1	1	Control virtual de las tres zonas.
2	Tira WS2812	2	Representación de Mecánica y Eléctrica.
3	Matriz WS2812 8x8	1	Símbolo de Zeus y panel central.
4	Resistencia de 330 Ω	3	Protección representativa de las líneas de datos.
5	Potenciómetro deslizante	2	Sonido simulado y ajuste de brillo.
6	Pulsador	3	Modo, paleta y secuencia coordinada.
7	OLED SSD1306	1	Visualización de modo, sonido, brillo y paleta.

N.º	Elemento	Cant.	Función
8	Monitor serial	1	Depuración y registro de eventos.

9 Distribución de pines y tendido de señales

Cuadro 5: Pinout de la simulación integrada

Función	GPIO	Conexión	Observación
Mecánica	16	GPIO – 330 Ω – DIN	Salida FastLED independiente
Centro	17	GPIO – 330 Ω – DIN	Matriz serpentina 8x8
Eléctrica	18	GPIO – 330 Ω – DIN	Salida FastLED independiente
Sonido	34	Cursor del potenciómetro	Se reemplaza por MAX4466 o micrófono digital
Brillo	35	Cursor del potenciómetro	Regulación global
Botón modo	25	Pulsador a GND	INPUT_PULLUP; tecla M
Botón color	26	Pulsador a GND	INPUT_PULLUP; tecla C
Botón secuencia	27	Pulsador a GND	INPUT_PULLUP; tecla S
OLED SDA	21	I2C	Dirección 0x3C
OLED SCL	22	I2C	Dirección 0x3C

9.1 Código de colores de cableado

Color recomendado	Señal	Aplicación
Rojo	Positivo de potencia	Batería, salida DC-DC y alimentación LED
Negro	GND	Retorno común dentro de cada módulo
Verde	Datos Mecánica	Salida del ESP32/driver hacia DIN
Azul	Datos centro	Señal hacia Zeus o matriz principal
Violeta	Datos Eléctrica	Señal hacia el panel derecho
Amarillo/naranja	Entradas analógicas	Micrófono, LDR y medición de batería
Blanco	I2C o señal auxiliar	OLED y sensores

9.2 Reglas de tendido físico

- No hacer circular la corriente de las tiras a través del pin 5 V del ESP32.
- Mantener la línea de datos corta entre el driver y el primer LED.
- Colocar la resistencia de 330–470 Ω cerca de la salida del driver.
- Colocar un condensador de 470–1000 μF cerca del inicio de cada tira.
- Realizar inyección de 5 V y GND en varios puntos cuando la tira sea larga.
- Separar físicamente cables de potencia y señales cuando sea posible.
- Incorporar alivio de tensión y conectores con seguro para resistir vibraciones.

10 Modos de iluminación

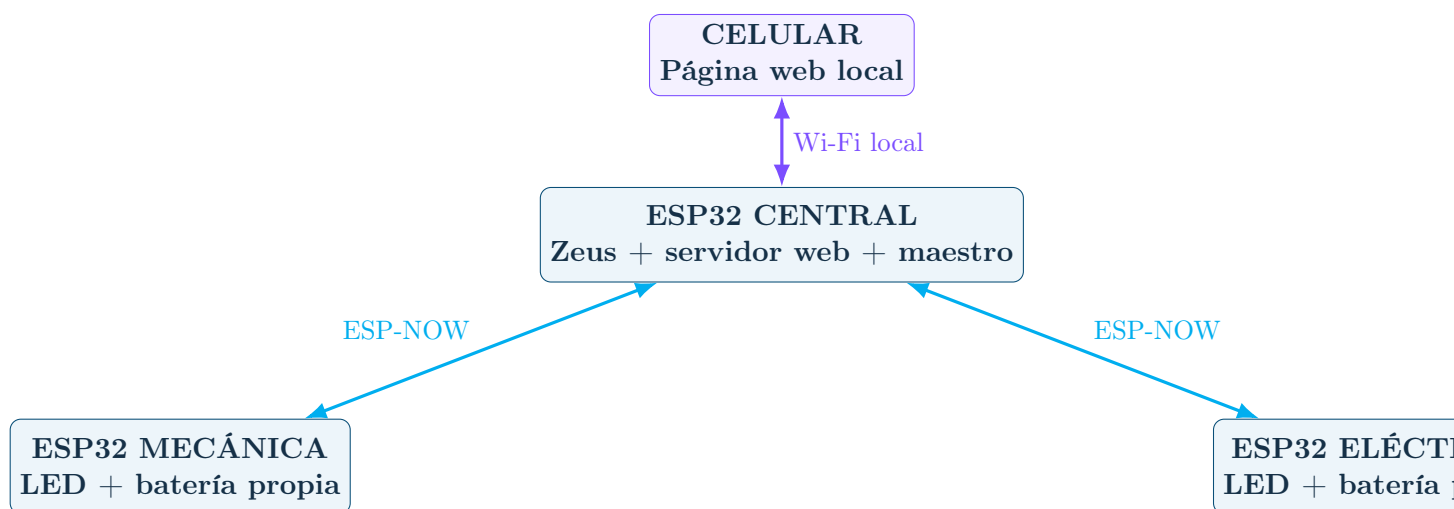
Cuadro 6: Catálogo de modos programados y propuestos

N.º	Modo	Descripción	Uso
1	Institucional	Colores mecánico y eléctrico, rayo central permanente y barrido suave	Marcha normal
2	Flujo IME	Mecánica carga energía, Zeus se activa y Eléctrica recibe la descarga	Presentación principal
3	Audio / VU	Cantidad de LED proporcional al nivel de sonido	Música, aplausos y bombo
4	Tormenta Zeus	Chispas, descargas y destellos controlados	Momento de impacto
5	Desfile	Secuencia multicolor continua	Recorrido festivo
6	Apagado	Todos los LED apagados	Ahorro y seguridad
7	Seguro	Efecto suave autónomo si se pierde la comunicación	Respaldo
8	Identificación	Cada módulo ilumina su nombre y color principal	Prueba y diagnóstico

10.1 Secuencia principal propuesta

1. Encendido ascendente de MECÁNICA.
2. Simulación de movimiento de engranajes.
3. Pulso luminoso desde Mecánica hacia el centro.
4. Activación progresiva del rayo de Zeus.
5. Destello central breve y limitado.
6. Descarga del centro hacia ELÉCTRICA.
7. Encendido de ELÉCTRICA con barrido azul/cian.
8. Iluminación final conjunta con el nombre institucional.

11 Arquitectura física final de tres ESP32



11.1 Razón para utilizar ESP-NOW

ESP-NOW permite comunicación directa entre dispositivos ESP32 sin router, con baja sobrecarga y envío de comandos pequeños. El maestro no debe transmitir el estado individual de cada LED; solamente enviará modo, brillo, color, velocidad, secuencia y hora de inicio. Cada receptor ejecutará localmente sus efectos. [10]

11.2 Estructura del mensaje

Cuadro 7: Campos principales del protocolo propuesto

Campo	Tipo	Función
Firma	uint32	Identifica mensajes válidos del proyecto IME
Secuencia	uint16	Detecta mensajes repetidos o perdidos
Destino	uint8	Todos, Mecánica, centro o Eléctrica
Modo	uint8	Selecciona la animación
Brillo	uint8	Límite de 0 a 100 %
Velocidad	uint8	Velocidad relativa de la animación
RGB	3 × uint8	Color base
EjecutarEnMs	uint32	Hora futura para sincronizar los tres nodos
CRC simple	uint16	Verificación preliminar de integridad

11.3 Control desde el celular

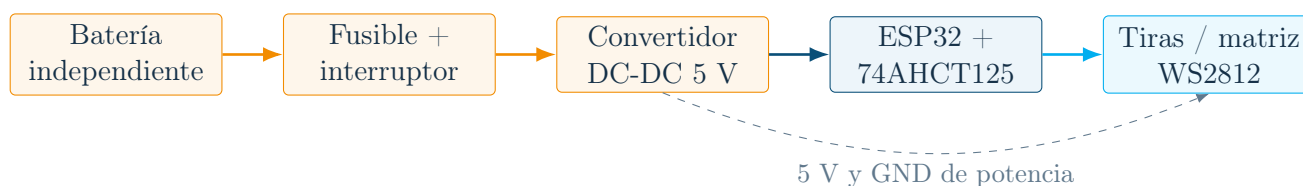
El ESP32 central funcionará como punto de acceso Wi-Fi con una red local, por ejemplo ESTANDARTE_IME. El operador se conectará desde el teléfono y abrirá la dirección 192.168.4.1. La interfaz permitirá:

- encendido y apagado general;
- selección de modo y color;
- control de brillo y velocidad;

- activación individual de Mecánica, Zeus o Eléctrica;
- ejecución de la secuencia general;
- visualización de conexión y batería;
- apagado de emergencia.

12 Sistema de potencia y baterías

12.1 Arquitectura por módulo



12.2 Ecuaciones de dimensionamiento

$$I_{\text{máx}} = N_{LED} I_{LED,\text{máx}} \quad (1)$$

$$P_{\text{máx}} = V_{LED} I_{\text{máx}} \quad (2)$$

$$E_{req} = P_{prom} t \quad (3)$$

$$C_{Ah} = \frac{E_{req}}{V_{bat} \eta} \quad (4)$$

Donde η es la eficiencia total del convertidor y del cableado. Para el diseño final se deberá medir el consumo del efecto real y no depender únicamente del máximo teórico.

12.3 Escenarios preliminares

Cuadro 8: Escenarios referenciales de corriente a 5 V

LED	Máx. a 60 mA/LED	Escenario 20 mA/LED	Límite recomendado inicial	Uso probable
72	4,32 A	1,44 A	2,5–3,5 A	Panel lateral
100	6,00 A	2,00 A	3,5–4,5 A	Lateral grande
144	8,64 A	2,88 A	4,5–6,0 A	Centro medio
200	12,00 A	4,00 A	6,0–8,0 A	Centro grande

- Fusible individual por módulo.
- Interruptor general accesible.
- BMS adecuado si se utilizan baterías de litio.

- Convertidor DC-DC sobredimensionado y ventilado.
- Cable y conectores dimensionados para la corriente.
- Protección contra polaridad invertida.
- Caja cerrada, pero con disipación térmica.
- Carga de baterías fuera del estandarte o mediante sistema certificado.

12.4 Opciones de batería a evaluar

Opción	Ventajas	Consideraciones
LiFePO4 12,8 V	Mayor estabilidad térmica, buena vida útil, tensión adecuada para DC-DC	Mayor costo y volumen según capacidad
Li-ion 3S con BMS	Buena relación energía/peso y disponibilidad	Requiere BMS, cargador correcto y protección mecánica
Power bank USB-C PD	Fácil carga y transporte	Debe verificarse corriente sostenida, perfil de salida y apagado automático
Batería sellada 12 V	Robusta y simple	Mayor peso; poco conveniente para módulos portátiles

13 Diseño mecánico y distribución física

13.1 Criterios de diseño

- Bajo peso y centro de gravedad próximo al portador.
- Batería ubicada en la zona inferior.
- Paneles desmontables y accesibles para mantenimiento.
- Difusor delante de los LED para evitar puntos luminosos agresivos.
- Canales protegidos para cableado.
- Caja electrónica separada de la zona de agarre.
- Protección contra golpes y movimiento repetitivo.
- Ventilación para convertidores y controladores.

13.2 Materiales candidatos

Material	Aplicación	Observación
Tubo de aluminio	Bastidor y soporte	Ligero y resistente
PVC espumado	Panel frontal y letras	Fácil de cortar y ligero
Polycarbonato/acrílico	Difusores y ventanas	Probar resistencia y peso
Tela impresa	Fondo gráfico	Reduce peso y permite reemplazo
Impresión 3D	Soportes, cajas y guías	Útil para piezas pequeñas
Canaleta plástica	Protección de cableado	Facilita mantenimiento

14 Plan de trabajo integral

14.1 Estructura de desglose del trabajo

Cuadro 9: Fases, actividades y entregables

Fase	Nombre	Actividades principales	Entregable
0	Organización	Equipo, responsables, repositorio, calendario y restricciones	Acta de inicio
1	Diseño visual	Medidas, textos, Zeus, colores, distribución de LED y paneles	Boceto aprobado
2	Simulación integrada	Wokwi, pinout, OLED, controles, modos y monitor serial	Simulación funcional
3	Revisión visual	Ajuste de velocidad, brillo, secuencia y legibilidad	Catálogo aprobado
4	Desarrollo local	PlatformIO, organización del código y pruebas de compilación	Proyecto local
5	Red inalámbrica	ESP-NOW, página web, confirmaciones y modo seguro	Prueba de tres nodos
6	Ingeniería eléctrica	Corriente, batería, convertidores, fusibles, drivers y conectores	Esquemático y BOM
7	Prototipo parcial	20–30 LED, un ESP32, batería y prueba térmica	Prototipo de banco
8	Diseño mecánico	Bastidor, paneles, difusores, cajas y distribución de peso	Planos y modelo 3D
9	Construcción	Corte, montaje, cableado y fijaciones	Tres módulos ensamblados
10	Integración	Firmware final, sincronización, web y sensores	Sistema completo
11	Validación	Autonomía, movimiento, alcance, temperatura y emergencia	Informe de pruebas
12	Operación	Manual, carga, transporte, responsables y repuestos	Kit operativo

14.2 Cronograma referencial de 20 días

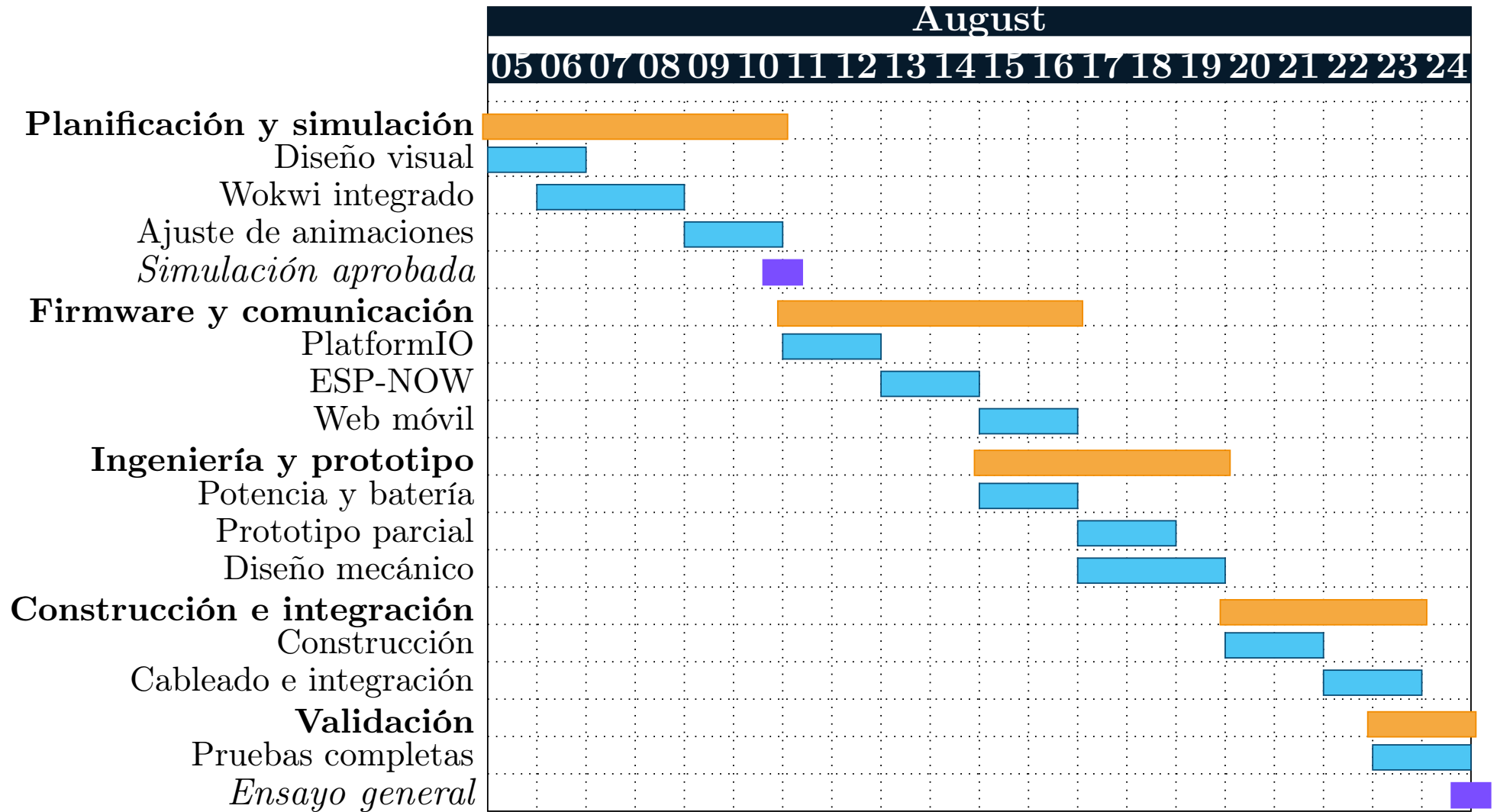


Figura 2: Cronograma referencial de 20 días, ajustado a la fecha final del 24 de agosto de 2026. Debe confirmarse con la fecha oficial y la disponibilidad del equipo.

14.3 Matriz RACI preliminar

Cuadro 10: Asignación de responsabilidades

Actividad	Coord.	Electrónica	Programación	Mecánica/diseño
Definición visual	A	C	C	R
Simulación Wokwi	A	C	R	C
Firmware ESP32	A	C	R	I
Potencia y protección	A	R	C	C
Estructura y soportes	A	C	I	R
Montaje y cableado	A	R	C	R
Pruebas	A	R	R	R
Operación en parada	A/R	C	C	C

R = Responsable; A = Aprueba; C = Consultado; I = Informado.

15 Lista preliminar de materiales

Cuadro 11: BOM preliminar para planificación

N.º	Componente	Cant.	Especificación inicial	Observación
1	ESP32 DevKit	4	Tres en servicio y uno de reserva	Mismo modelo para todos
2	Tira WS2812B	Por definir	5 V, densidad según diseño	Medir longitud real
3	Matriz/tiras para Zeus	1 conjunto	WS2812 o segmentos personalizados	Panel central
4	74AHCT125	3–4	Adaptación lógica 3,3 V a 5 V	Uno por módulo
5	Resistencia 330–470 Ω	3–6	En cada línea de datos	Cerca del driver
6	Condensador 1000 μ F	3–6	Baja ESR, tensión adecuada	Cerca de tiras
7	Convertidor DC-DC	3	Capacidad según corriente medida	Con ventilación
8	Batería	3	Capacidad según autonomía	Una por módulo
9	Fusible y portafusible	3–6	Dimensionado por módulo	Repuesto disponible
10	Interruptor general	3	Corriente suficiente	Accesible
11	MAX4466 o micrófono digital	1	Sensor del módulo central	Evaluar ruido
12	OLED SSD1306	1	I2C 0x3C	Diagnóstico local
13	Pulsadores	4–6	Modo, secuencia, emergencia	Panel central

N.º	Componente	Cant.	Especificación inicial	Observación
14	Conectores XT30/XT60 o equivalentes	Varios	Polarizados y seguros	Potencia
15	Conectores de señal	Varios	Bloqueables	Mantenimiento
16	Cable de potencia	Según diseño	Sección según corriente	Rojo/negro
17	Cable de señal	Según diseño	Flexible y protegido	Datos/sensores
18	Caja electrónica	3	Ventilada y desmontable	Una por módulo
19	Difusor	Según diseño	Acrílico, policarbonato o PVC	Uniformidad visual
20	Bastidor	3	Aluminio/PVC estructural	Peso bajo

15.1 Formato de presupuesto

Cuadro 12: Cuadro para completar cotizaciones

Partida	Cant.	P. unitario	Subtotal	Proveedor
Controladores y sensores				
LED y difusores				
Baterías y convertidores				
Protección y conectores				
Estructura mecánica				
Impresión, acabado y rotulado				
Repuestos y contingencia				
TOTAL				

16 Plan de pruebas y criterios de aceptación

Cuadro 13: Matriz de verificación

Prueba	Procedimiento	Criterio de aceptación
Compilación Wokwi	Ejecutar el firmware integrado	Sin errores ni reinicios
Zonas independientes	Activar cada salida	No hay interferencias
Botones	Pulsar repetidamente	Una acción por pulsación
Entradas analógicas	Mover potenciómetros	Respuesta progresiva
Secuencia	Ejecutar Flujo IME	Orden Mecánica–Zeus–Eléctrica

Prueba	Procedimiento	Criterio de aceptación
Brillo	Recorrer 10–100 %	Sin bloqueos ni cambios bruscos
Web móvil	Conectar celular y enviar comandos	Respuesta correcta y clara
ESP-NOW	Separar tres placas y enviar secuencias	Recepción estable y confirmada
Pérdida de enlace	Apagar el central	Laterales entran en modo seguro
Consumo	Medir corriente de cada modo	Dentro del límite configurado
Caída de tensión	Medir inicio y final de tiras	Diferencia aceptable, sin cambios de color
Temperatura	Operar el tiempo objetivo	Sin sobrecalentamiento
Autonomía	Ejecutar programa real	Cumple duración más margen
Vibración	Simular marcha	Sin desconexiones
Visibilidad diurna	Probar al exterior	Texto y efectos identificables
Emergencia	Activar apagado	Corte inmediato y seguro

16.1 Registro de resultados

Cada prueba deberá registrar fecha, responsable, firmware, batería, número de LED, modo, brillo, corriente, tensión, temperatura, resultado y observaciones. Los resultados deben almacenarse en una hoja compartida o dentro del repositorio.

17 Riesgos y medidas de control

Cuadro 14: Matriz preliminar de riesgos

Riesgo	Prob.	Impacto	Medida preventiva
Consumo mayor al previsto	Media	Alto	Limitar brillo, medir corriente y sobredimensionar convertidor
Caída de tensión	Alta	Medio	Inyección de alimentación y cable de mayor sección
Pérdida de comunicación	Media	Medio	Modo seguro y botones físicos
Polaridad invertida	Baja	Alto	Conectores polarizados y protección
Sobrecalentamiento	Media	Alto	Ventilación, disipación y prueba térmica
Vibración y desconexión	Media	Alto	Conectores con seguro y alivio de tensión
Autonomía insuficiente	Media	Alto	Prueba con programa real y batería con margen
Daño del ESP32	Baja	Medio	Placa de repuesto y conectores rápidos
Exceso de estrobo	Media	Medio	Limitar frecuencia, brillo y duración
Falla del celular	Media	Medio	Botones físicos y modo autónomo

Riesgo	Prob.	Impacto	Medida preventiva
Lluvia o humedad	Baja/Media	Alto	Protección mecánica y procedimiento de suspensión
Carga insegura de batería	Baja	Alto	Cargador apropiado y responsable designado

18 Organización del repositorio y control de versiones

Listing 2: Estructura del repositorio

```

1 Estandarte-LED-Interactivo-conIA/
2 |-- README.md
3 |-- INICIO.md
4 |-- documentacion/
5 |   |-- gran_plan/
6 |       |-- Gran_Plan_Trabajo_Simulacion_Estandarte_IME.tex
7 |       |-- Gran_Plan_Trabajo_Simulacion_Estandarte_IME.pdf
8 |   |-- informe/informe_final.md
9 |   |-- control/analisis_control.md
10 |-- simulacion_visual/
11 |   |-- Simulador_Visual_Estandarte_IME.html
12 |-- software/
13 |   |-- simulaciones/wokwi/
14 |       |-- sketch.ino
15 |       |-- diagram.json
16 |       |-- libraries.txt
17 |   |-- simulaciones/platformio/
18 |       |-- platformio.ini
19 |       |-- wokwi.toml
20 |       |-- diagram.json
21 |       |-- src/main.cpp
22 |   |-- firmware/arquitectura_3_esp32/
23 |       |-- comun/protocolo.h
24 |       |-- central/central_maestro.ino
25 |       |-- mecanica/receptor_mecanica.ino
26 |       |-- electrica/receptor_electrica.ino
27 |   |-- firmware/prototipo_IA/
28 |-- hardware/
29 |   |-- circuito/
30 |       |-- diagramas/esquematico.svg
31 |       |-- guias/montaje_pruebas_dias1-3.md
32 |-- recursos/
33 |   |-- captura_simulador.png
34 |   |-- codigos_QR (qr_wokwi, qr_github, qr_platformio, qr_espnw)

```

18.1 Reglas de trabajo

- Una rama para simulación, otra para red inalámbrica y otra para documentación.
- No subir contraseñas reales ni información sensible.
- Etiquetar versiones estables antes de las pruebas físicas.
- Registrar cambios de pines, número de LED y configuración de batería.
- Mantener un ESP32 de repuesto con la última versión estable.

19 Manual operativo preliminar

19.1 Antes de la parada

1. Cargar las tres baterías y verificar tensión.
2. Revisar fusibles, conectores y fijaciones.
3. Encender cada módulo individualmente.
4. Verificar el modo seguro de cada lateral.
5. Encender el central y conectar el celular.
6. Ejecutar una secuencia general.
7. Confirmar autonomía y llevar repuestos.

19.2 Durante la parada

- Mantener brillo moderado durante el recorrido normal.
- Reservar el modo Tormenta para momentos breves.
- Evitar modificar el cableado con el sistema energizado.
- Designar una sola persona para el control desde celular.
- Tener acceso inmediato a los interruptores generales.

19.3 Después de la actividad

- Apagar los módulos antes de desconectar baterías.
- Revisar temperatura, conectores y cables.
- Cargar baterías según el procedimiento del fabricante.
- Registrar fallas y mejoras para el siguiente uso.

20 Entregables y criterios de cierre

El proyecto se considerará listo para la parada cuando cumpla lo siguiente:

1. Simulación integrada aprobada por el equipo.
2. Diseño visual y dimensiones cerradas.
3. Tres ESP32 sincronizados mediante ESP-NOW.
4. Control web accesible desde al menos dos teléfonos de prueba.
5. Consumo medido y limitado por módulo.
6. Autonomía comprobada con margen.
7. Estructura resistente al movimiento.
8. Modo seguro y apagado de emergencia verificados.
9. Manual, repuestos y responsables definidos.

21 Conclusiones y decisión recomendada

La ruta más segura y eficiente es comenzar con Wokwi y el simulador visual local, aprobar la apariencia y los modos, y recién después cerrar el número de LED y el sistema de baterías. La arquitectura final debe usar tres ESP32, ESP-NOW entre módulos y una página web local alojada en el ESP32 central.

Abrir `simulacion_visual/Simulador_Visual_Estandarte_IME.html`, revisar la secuencia Mecánica–Zeus–Eléctrica y luego importar la carpeta `wokwi_integrado` en Wokwi. Después de aprobar los efectos, se debe definir la cantidad real de LED y construir un prototipo de 20–30 LED.

A Anexo A: archivo PlatformIO

Listing 3: platformio.ini

```
1 [platformio]
2 default_envs = esp32dev
3
4 [env:esp32dev]
5 platform = espressif32
6 board = esp32dev
7 framework = arduino
8 monitor_speed = 115200
9 lib_deps =
10     fastled/FastLED
11     adafruit/Adafruit GFX Library
12     adafruit/Adafruit SSD1306
13 build_flags =
14     -DCORE_DEBUG_LEVEL=1
```

B Anexo B: configuración Wokwi local

Listing 4: wokwi.toml

```
1 [wokwi]
2 version = 1
3 firmware = '.pio/build/esp32dev/firmware.bin'
4 elf = '.pio/build/esp32dev/firmware.elf'
5
6 # Para una fase posterior con servidor web en el ESP32 virtual:
7 # [[net.forward]]
8 # from = 'localhost:8180'
9 # to = 'target:80'
```

C Anexo C: protocolo común de tres ESP32

Listing 5: protocolo.h

```
1 #pragma once
2 #include <Arduino.h>
3
4 constexpr uint32_t FIRMA_IME = 0x494D4532; // "IME2"
5 constexpr uint8_t VERSION_PROTOCOLO = 1;
```

```

6
7 enum class Destino : uint8_t {
8     TODOS = 0,
9     MECANICA = 1,
10    CENTRO = 2,
11    ELECTRICA = 3
12 };
13
14 enum class ModoIME : uint8_t {
15     INSTITUCIONAL = 0,
16     FLUJO = 1,
17     AUDIO_VU = 2,
18     TORMENTA = 3,
19     DESFILE = 4,
20     APAGADO = 5,
21     SEGURO = 6
22 };
23
24 struct __attribute__((packed)) ComandoIME {
25     uint32_t firma = FIRMA_IME;
26     uint16_t secuencia = 0;
27     uint8_t version = VERSION_PROTOCOLO;
28     Destino destino = Destino::TODOS;
29     ModoIME modo = ModoIME::INSTITUCIONAL;
30     uint8_t brillo = 80;
31     uint8_t v velocidad = 50;
32     uint8_t rojo = 0;
33     uint8_t verde = 174;
34     uint8_t azul = 239;
35     uint32_t ejecutarEnMs = 0;
36     uint16_t crcSimple = 0;
37 };
38
39 struct __attribute__((packed)) EstadoIME {
40     uint32_t firma = FIRMA_IME;
41     uint8_t nodo = 0;
42     uint16_t ultimaSecuencia = 0;
43     uint16_t bateriaMv = 0;
44     int8_t rssi = 0;
45     uint8_t errores = 0;
46     uint32_t uptimeS = 0;
47 };
48
49 inline uint16_t calcularCrcSimple(const ComandoIME &c) {
50     const uint8_t *p = reinterpret_cast<const uint8_t *>(&c);
51     uint16_t suma = 0;
52     for (size_t i = 0; i < sizeof(ComandoIME) - sizeof(c.crcSimple); ++i) suma = (suma << 1) ^ p[i];
53     return suma;
54 }
55
56 inline bool comandoValido(const ComandoIME &c) {
57     return c.firma == FIRMA_IME && c.version == VERSION_PROTOCOLO && c.crcSimple == calcularCrcSimple(c);
58 }

```

D Anexo D: firmware del ESP32 central

Listing 6: Inicio y dependencias del firmware central

```

1 #include <Arduino.h>
2 #include <WiFi.h>
3 #include <WebServer.h>
4 #include <esp_now.h>

```



```

5 #include "../comun/protocolo.h"
6
7 // Reemplazar con las MAC reales obtenidas de cada ESP32 receptor.
8 uint8_t MAC_MECANICA[] = {0x24,0x6F,0x28,0x00,0x00,0x01};
9 uint8_t MAC_ELECTRICA[] = {0x24,0x6F,0x28,0x00,0x00,0x02};
10 constexpr uint8_t CANAL_WIFI = 6;
11
12 WebServer servidor(80);
13 ComandoIME comando;
14 uint16_t secuencia = 0;

```

El archivo completo se incluye en el paquete ZIP.

E Anexo E: receptor Mecánica

Listing 7: Firmware receptor Mecánica

```

1 #include <Arduino.h>
2 #include <WiFi.h>
3 #include <esp_now.h>
4 #include <FastLED.h>
5 #include "../comun/protocolo.h"
6
7 constexpr uint8_t PIN_LED = 16;
8 constexpr uint16_t NUM_LED = 72; // ajustar al diseño real
9 CRGB leds[NUM_LED];
10 ComandoIME pendiente;
11 volatile bool nuevoComando = false;
12 uint32_t ultimaRecepcion = 0;
13
14 void recibir(const esp_now_recv_info_t*, const uint8_t *datos, int len) {
15     if (len != sizeof(ComandoIME)) return;
16     memcpy(&pendiente, datos, sizeof(pendiente));
17     if (comandoValido(pendiente)) { nuevoComando = true; ultimaRecepcion = millis(); }
18 }
19
20 void aplicarModo(const ComandoIME &c) {
21     FastLED.setBrightness(map(c.brillo, 0, 100, 0, 180));
22     switch (c.modo) {
23         case ModoIME::APAGADO: fill_solid(leds, NUM_LED, CRGB::Black); break;
24         case ModoIME::TORMENTA: fill_solid(leds, NUM_LED, CRGB(255,100,0)); break;
25         default: fill_rainbow(leds, NUM_LED, millis()/20, 4); break;
26     }
27     FastLED.show();
28 }
29
30 void setup() {
31     Serial.begin(115200);
32     WiFi.mode(WIFI_STA);
33     WiFi.setChannel(6);
34     FastLED.addLeds<WS2812B, PIN_LED, GRB>(leds, NUM_LED);
35     FastLED.setMaxPowerInVoltsAndMilliamps(5, 3500);
36     if (esp_now_init() != ESP_OK) ESP.restart();
37     esp_now_register_recv_cb(recibir);
38 }
39
40 void loop() {
41     if (nuevoComando && (int32_t)(millis() - pendiente.ejecutarEnMs) >= 0) {
42         nuevoComando = false;
43         aplicarModo(pendiente);
44     }
45     if (millis() - ultimaRecepcion > 5000) {
46         // Modo seguro si se pierde el enlace.

```

```
47     fadeToBlackBy(leds, NUM_LED, 8);
48     leds[(millis()/100)%NUM_LED] = CRGB(255,90,0);
49     FastLED.show();
50 }
51 delay(10);
52 }
```

F Anexo F: receptor Eléctrica

Listing 8: Firmware receptor Eléctrica

```
1  #include <Arduino.h>
2  #include <WiFi.h>
3  #include <esp_now.h>
4  #include <FastLED.h>
5  #include "../comun/protocolo.h"
6
7  constexpr uint8_t PIN_LED = 18;
8  constexpr uint16_t NUM_LED = 72; // ajustar al diseño real
9  CRGB leds[NUM_LED];
10 ComandoIME pendiente;
11 volatile bool nuevoComando = false;
12 uint32_t ultimaRecepcion = 0;
13
14 void recibir(const esp_now_recv_info_t*, const uint8_t *datos, int len) {
15     if (len != sizeof(ComandoIME)) return;
16     memcpy(&pendiente, datos, sizeof(pendiente));
17     if (comandoValido(pendiente)) { nuevoComando = true; ultimaRecepcion = millis(); }
18 }
19
20 void aplicarModo(const ComandoIME &c) {
21     FastLED.setBrightness(map(c.brillo, 0, 100, 0, 180));
22     switch (c.modo) {
23         case ModoIME::APAGADO: fill_solid(leds, NUM_LED, CRGB::Black); break;
24         case ModoIME::TORMENTA:
25             fill_solid(leds, NUM_LED, CRGB::Black);
26             for (uint8_t i=0; i<8; i++) leds[random16(NUM_LED)] = CRGB::White;
27             break;
28         default: fill_rainbow(leds, NUM_LED, millis()/18 + 120, 4); break;
29     }
30     FastLED.show();
31 }
32
33 void setup() {
34     Serial.begin(115200);
35     WiFi.mode(WIFI_STA);
36     WiFi.setChannel(6);
37     FastLED.addLeds<WS2812B, PIN_LED, GRB>(leds, NUM_LED);
38     FastLED.setMaxPowerInVoltsAndMilliamps(5, 3500);
39     if (esp_now_init() != ESP_OK) ESP.restart();
40     esp_now_register_recv_cb(recibir);
41 }
42
43 void loop() {
44     if (nuevoComando && (int32_t)(millis() - pendiente.ejecutarEnMs) >= 0) {
45         nuevoComando = false;
46         aplicarModo(pendiente);
47     }
48     if (millis() - ultimaRecepcion > 5000) {
49         fadeToBlackBy(leds, NUM_LED, 8);
50         leds[NUM_LED-1-(millis()/100)%NUM_LED] = CRGB(0,150,255);
51         FastLED.show();
52     }
```

```
53   delay(10);  
54 }
```

G Anexo G: enlaces oficiales

Referencias

- [1] Wokwi, *Welcome to Wokwi*. <https://docs.wokwi.com/>
- [2] Wokwi, *ESP32 Simulation*. <https://docs.wokwi.com/guides/esp32>
- [3] Wokwi, *Frequently Asked Questions*. <https://docs.wokwi.com/faq>
- [4] Wokwi, *WS2812 LED Strip Reference*. <https://docs.wokwi.com/parts/wokwi-led-strip>
- [5] Wokwi, *WS2812 LED Matrix Reference*. <https://docs.wokwi.com/parts/wokwi-led-matrix>
- [6] Wokwi, *diagram.json File Format*. <https://docs.wokwi.com/diagram-format>
- [7] Wokwi, *Getting Started with Wokwi for VS Code*. <https://docs.wokwi.com/vscode/getting-started>
- [8] Wokwi, *Configuring Your Project*. <https://docs.wokwi.com/vscode/project-config>
- [9] PlatformIO, *PlatformIO IDE for VSCode*. <https://docs.platformio.org/en/stable/integration/ide/vscode.html>
- [10] Espressif Systems, *Arduino-ESP32 ESP-NOW API*. <https://docs.espressif.com/projects/arduino-esp32/en/latest/api/espnow.html>
- [11] Espressif Systems, *Arduino-ESP32 Wi-Fi API*. <https://docs.espressif.com/projects/arduino-esp32/en/latest/api/wifi.html>
- [12] FastLED, *Power Management Functions*. https://fastled.io/docs/d3/d1d/group___power.html
- [13] Gael Renzo, *Estandarte LED Interactivo con IA*. <https://github.com/gaelrenzo/Estandarte-LED-Interactivo-conIA>